STUDENT NOTES HUB & AI TUTOR

Comprehensive System Architecture & Technical Specification

Full-Stack Node.js/Express * React 19 * Google Gemini 3.6 Flash * ChromaDB Cloud Vector RAG

DOCUMENT INFORMATION & METADATA

Project: Interactive Course Notes & Smart AI Tutor Platform
Authors: AI Engineering Team & System Architects
Runtime Environment: Node.js 22 (LTS) | React 19 SPA | Express Server Engine
Vector Database: ChromaDB Managed Cloud Client ('rag-notes-application1')
Primary Generative Model: Google Gemini 3.6 Flash (@google/genai SDK)
Document Format Compatibility: PDF (Vector/Canvas), Word (.docx), PowerPoint (.pptx), Markdown, Code (.py)
Status: Production Ready | Fully Tested & Deployed

Executive Overview

The Student Notes Hub & AI Tutor is an enterprise-grade academic platform engineered to bridge the gap between static lecture materials and active, personalized learning. Built on modern TypeScript, it unifies dual operational portals: a feature-packed Teacher & Instructor Studio for curriculum compilation, slide extraction, and vector database indexing, alongside an immersive Student Study Hub featuring side-by-side note reading, text-to-speech, interactive self-assessment exams, and a context-grounded AI Tutor.

Unlike generic LLM wrappers, this system implements a strict Retrieval-Augmented Generation (RAG) architecture. When students ask questions or generate exam questions, the engine grounds prompts against exact slide sections and page citations indexed in ChromaDB Cloud, guaranteeing factual fidelity and preventing hallucinations.

Key Technical Capabilities

- **High-Fidelity Dual-View Reading:** Presents original slide diagrams, equations, and figures via a hardware-accelerated PDF canvas viewer synchronized side-by-side with structured, readable markdown notes.
- **Real-Time Multi-Provider AI Routing:** Prioritizes Google Gemini 3.6 Flash for sub-second responses with seamless automatic fallback to OpenRouter (Llama 3.3 70B, DeepSeek) and local semantic heuristic engines.
- **ChromaDB Cloud Vector Indexing:** Deconstructs multi-megabyte lecture presentations into clean, semantically embedded topic units with metadata tracking document IDs, subjects, and page numbers.
- **Automated Practice Exam Generation:** Instantly compiles balanced, 4-choice multiple-choice quizzes complete with verified answer keys, concept references, and rationale explanations.

1. System Architecture & High-Level Design

The application follows a clean full-stack decoupled architecture. The Express server acts as both the secure API gateway and the static asset distributor, running Vite in middleware mode during development and serving compiled production assets in deployment.

> **High-Level Data Flow Architecture**
>
> [Client Layer: React 19 SPA]
> |-- Student Portal (Viewer, AI Side Panel, Audio Player, Quiz Engine)
> |-- Teacher Studio (Document Uploader, Metadata Editor, Announcement Hub)
> |
> v [HTTP / JSON REST API with Authorization & Session State]
> [Backend Layer: Express 4.x + Node.js 22 Engine]
> |-- File Parsing Engine (pdf-parse, mammoth, adm-zip for pptx)
> |-- Chroma Vector Sync Service (CloudClient / @chroma-core/chroma-client)
> |-- Multi-Provider AI Router (Gemini 3.6 Flash -> OpenRouter -> Groq -> Local RAG)
> |-- Storage Engine (Persistent JSON file cache + Server Sync Gateway)

2. Technology Stack & Framework Specifications

Every dependency was selected for zero-friction performance, strict type safety, and minimal container footprint:

Layer / Component	Technology / Library	Version	Operational Role
Frontend Framework	React + TypeScript	19.0.1	Single-Page Application with Concurrent React
Styling & Tokens	Tailwind CSS + Lucide	v4.1.14	Responsive dark-first UI with SVG icon set
PDF Rendering	PDF.js (pdfjs-dist)	6.2.108	Canvas rasterizer for original lecture slides
Backend Framework	Express.js (Node.js)	4.21.2	REST API, file streaming, and AI routing
Primary AI Model	Google Gemini 3.6 Flash	v2.4.0	Core RAG Tutor & structured quiz generator
Vector Database	ChromaDB Cloud Client	3.5.0	Cloud vector storage for slide embeddings
Office Doc Parsers	Mammoth & Adm-Zip	1.12.2	DOCX and PPTX slide XML deconstruction
PDF Ingestion	pdf-parse	2.4.5	Raw text, page split, and metadata extraction

3. Dual-Portal Deployment Architecture

The application natively supports three distinct deployment configurations via environment variables or URL query routing:

- **Unified Mode (Default):** Both the Teacher Studio and Student Portal exist in the same instance, allowing administrators to toggle portals seamlessly via header navigation or URL parameters.
- **Standalone Student Mode (VITE_PORTAL_MODE=student):** Locks the interface into a secure, read-only student experience. All administrative studio endpoints and editor buttons are stripped from the DOM.
- **Standalone Teacher Mode (VITE_PORTAL_MODE=teacher):** Serves only the curriculum management studio. Used in institutional setups where instructors manage notes behind internal university VPNs.

4. AI & Retrieval-Augmented Generation (RAG) Subsystem

The hallmark of the platform is its grounded intelligence pipeline. Rather than dumping raw text into a single prompt, the system utilizes a multi-step semantic retrieval workflow:

- **Step 1: Document Chunking:** When a note is ingested, the parser decomposes content into discrete slide/page sections. Each section retains its page number, topic heading, and extracted bullet text.
- **Step 2: Vector Embedding & Storage:** Document chunks are dispatched to ChromaDB Cloud via @chroma-core/chroma-client. Each record is assigned an embedding vector alongside metadata tags (documentId, subject, unit, pageNumber).
- **Step 3: Query Analysis & Similarity Search:** When a student submits a query, the backend converts the query to an embedding, executes a cosine similarity search across the active document's collection, and extracts the top 5 most relevant slide excerpts.
- **Step 4: Grounded Prompt Assembly:** The system constructs a strict system prompt instructing the model to act as an academic tutor, cite page numbers, and explicitly refuse to invent information outside the syllabus.
- **Step 5: Multi-Tier Execution & Fallback:** Requests are dispatched first to Google Gemini 3.6 Flash. If quota or network limits are encountered, the engine transparently retries against OpenRouter (Llama 3.3 70B), Groq, or the internal keyword matcher.

> **Grounded Prompt Template (System Instruction)**
>
> You are an expert university academic tutor specialized in course materials.
> - Ground all explanations directly on the retrieved slide excerpts.
> - Always cite specific slide/page numbers (e.g., [Page 4 - Virtualization Models]).
> - Format answers using clean markdown, structured bullet points, and code blocks.
> - If the question cannot be answered from the provided notes, politely explain that the concept is not covered in the current syllabus.

5. Automated Quiz & Assessment Engine

Students can generate customized practice exams for any uploaded lecture document. The backend leverages Gemini's structured responseMimeType: 'application/json' to enforce a strict JSON schema:

Field Name	Type	Description & Validation
id	string	Unique identifier formatted as `gemini-{docId}-{timestamp}-{index}`
question	string	Conceptually rigorous question stem derived from lecture slides
options	string[]	Exactly 4 plausible choices (1 correct, 3 realistic distractors)
correctIndex	number	0-indexed pointer (0 to 3) referencing the correct option
explanation	string	Comprehensive breakdown explaining why the answer is correct
topic	string	The specific curriculum unit or lecture concept tested

6. Multi-Format Document Ingestion Engine

Instructors can upload course materials in multiple industry-standard formats. The ingestion pipeline normalizes all documents into a standardized NoteDocument schema:

- **PDF Files (.pdf):** Processed via pdf-parse to extract clean per-page text and headers. Raw PDF binary is preserved as a Data URL or static stream to allow high-fidelity canvas rendering in the viewer.
- **Word Documents (.docx, .doc):** Parsed using Mammoth to extract raw text, table structures, and headings while preserving paragraph flow.
- **PowerPoint Presentations (.pptx):** Decompressed via adm-zip to parse slide XML structures (`ppt/slides/slide{N}.xml`), extracting bullet text and slide speaker notes per slide.
- **Source Code Files (.py, .ts, .js, .java):** Segmented by function and class definitions, wrapped in syntax-highlighted code blocks, and indexed for programming coursework.

7. High-Fidelity UI & Document Viewer Architecture

The DocumentModalViewer component offers a desktop-grade study environment optimized for long reading sessions:

- **Synchronized Dual View:** Displays the original presentation slide PDF on the left and enhanced, accessible AI markdown notes on the right.
- **Page Thumbnails & Jump Navigation:** Interactive slide drawer with instant thumbnail navigation, active page indicator, and page count status.
- **Embedded AI Tutor Side Panel:** A docked right-hand panel allowing students to converse with the AI tutor without ever losing their place in the lecture notes.
- **Text-to-Speech (TTS) Audio Player:** Integrated speech synthesis player with play, pause, playback speed control (0.75x to 2.0x), and automatic page progression.
- **Interactive Bookmarks & Highlights:** Students can bookmark critical slides with custom annotations, stored locally for instant retrieval across sessions.

> **Supported Document Operations (Teacher Studio)**
> - Upload & Auto-Vectorize: Automatic text extraction and vector embedding generation.
> - Document Merging: Combine multiple topic slides into comprehensive course modules.
> - Rename & Metadata Tagging: Modify course codes, units, titles, and descriptions.
> - Publish / Unpublish Toggle: Control student visibility for draft materials.
> - Course Announcements: Post high-priority exam dates and syllabus updates.

8. Complete REST API Endpoint Reference

Method	Endpoint	Payload / Parameters	Description
GET	/api/documents	None	Returns list of all published or uploaded documents
POST	/api/documents	{ title, subject, unit, pdfUrl, sections }	Uploads and persists a new course document
PUT	/api/documents/:id	{ title, subject, unit, isPublished }	Updates document metadata or publication state
DELETE	/api/documents/:id	None	Permanently deletes document and vector indexes
POST	/api/chat	{ prompt, context, activeDocument }	Executes grounded RAG chat against notes
POST	/api/generate-quiz	{ docId, documentTitle, content, count }	Compiles a 4-choice practice exam from notes
POST	/api/parse-document	Multipart/Form-Data (file)	Parses PDF/DOCX/PPTX into structured sections
GET	/api/announcements	None	Returns active broadcast announcements
POST	/api/announcements	{ title, content, priority }	Posts a new course announcement
GET	/api/chroma/status	None	Checks ChromaDB cloud vector database health
POST	/api/sync-with-teacher	{ teacherUrl }	Syncs student instance with remote teacher deployment

9. Environment Configuration & Deployment

The application is container-ready and deploys effortlessly to Cloud Run, Render, or Docker:

Recommended Environment Variables (.env)

```
GEMINI_API_KEY=your_gemini_api_key # Google Gemini 3.6 Flash access
CHROMA_API_KEY=your_chroma_key # ChromaDB Cloud authentication
CHROMA_DATABASE=default_database # Chroma database target
OPENROUTER_API_KEY=optional_key # Fallback LLM provider
GROQ_API_KEY=optional_key # High-speed fallback provider
VITE_PORTAL_MODE=student|teacher # Optional hard portal lock
PORT=3000 # Application HTTP port
```

To build and launch the production container, execute:

1. npm install --production=false
2. npm run build (compiles frontend Vite bundle & bundles server.ts via esbuild to dist/server.cjs)
3. npm start (launches compiled CommonJS server on port 3000)